RNN, LSTM, Transformers Building up towards Generative AI

Advanced Natural Language Processing

Sher Muhammad Daudpota

Sukkur IBA University

Code: Text Vectorization

Does the sequence of word matter in text?

Drink _____?

Does the sequence of word matter?

Irfan has a cat and she loves to drink_____?

Does the sequence of word matter?

What should be after p_ in a word?

Does the sequence of word matter?

Ap___?

What should be after p in above word?

Recurrent Neural Network: where sequence is regarded highly

“working love learning we on deep”

Does it make sense?

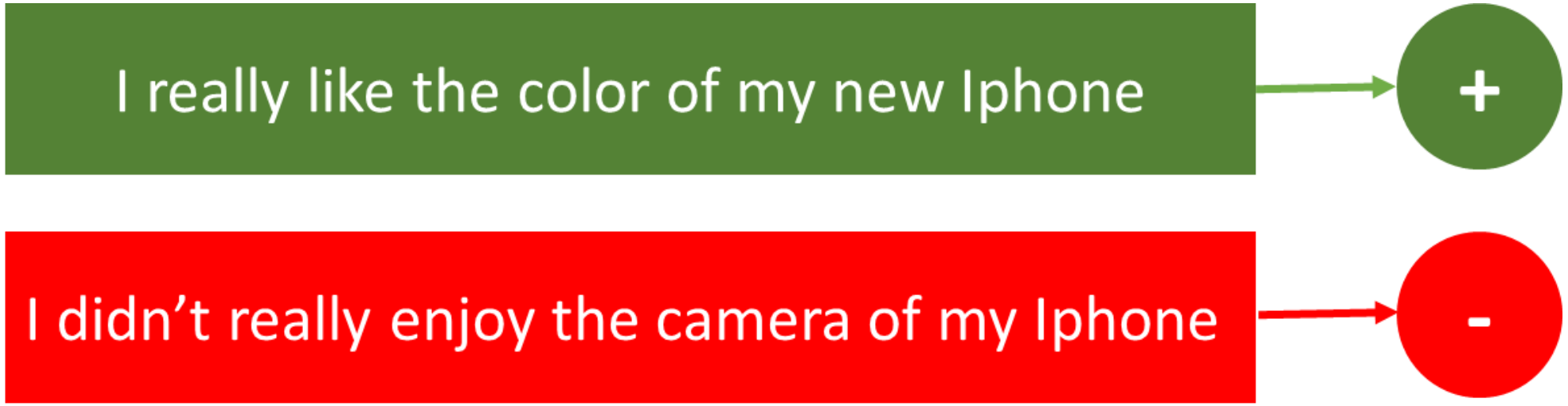
Recurrent Neural Network: where sequence is regarded highly

More sensible, isn’t?

“we love working on deep learning”

When words are in correct sequence

RNN- Sentiment Classification



RNN: Image Captioning

A person riding a motorcycle on a dirt road.

Two dogs play in the grass.

A group of young people playing a game of frisbee.

Two hockey players are fighting over the puck.

Language Translation

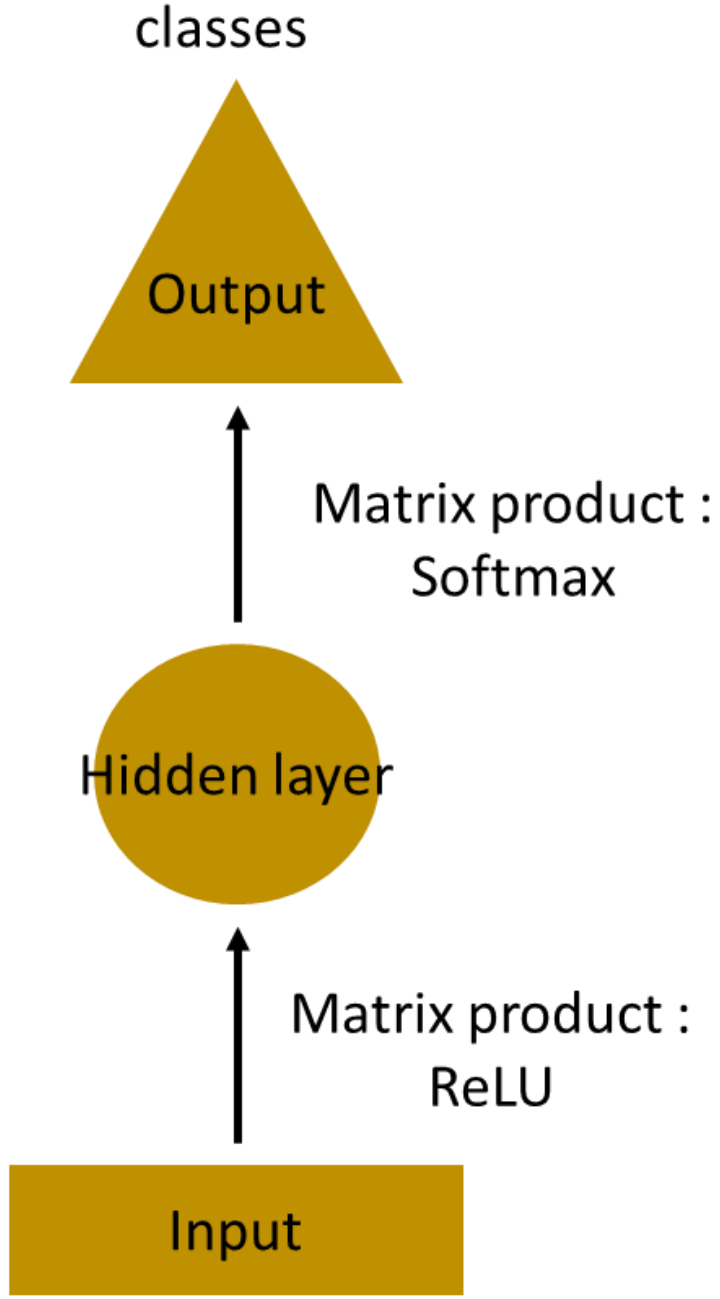
French was the official language of the colony of French Indochina, comprising modern-day Vietnam, Laos, and Cambodia. It continues to be an administrative language in Laos and Cambodia, although its influence has waned in recent years.

Translate into : French

Le français était la langue officielle de la colonie de l'Indochine française, comprenant le Vietnam d'aujourd'hui, le Laos et le Cambodge. Il continue d'être une langue administrative au Laos et au Cambodge, bien que son influence a décliné au cours des dernières années.

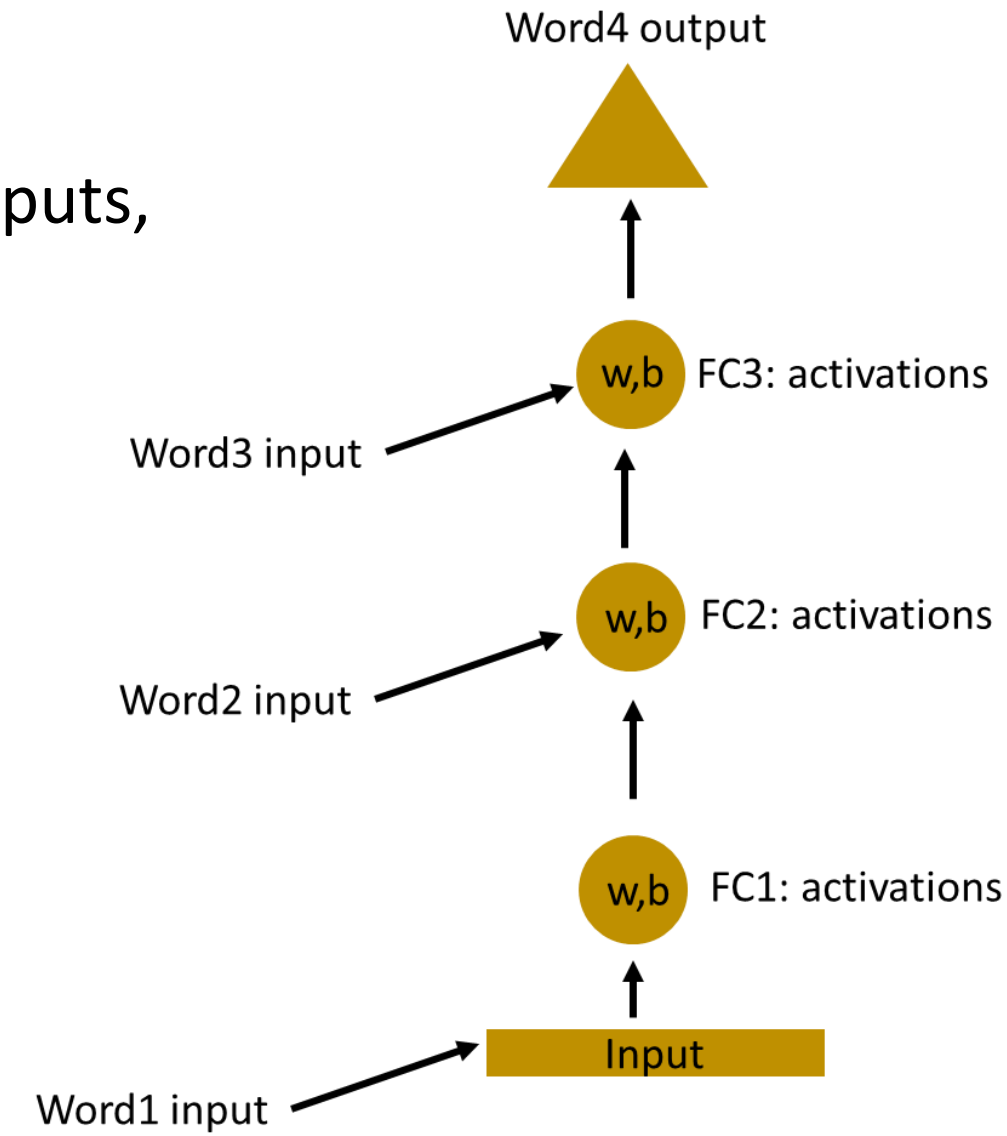
So, what's RNN?

- Suppose next word predication, here is what MLP would look like,



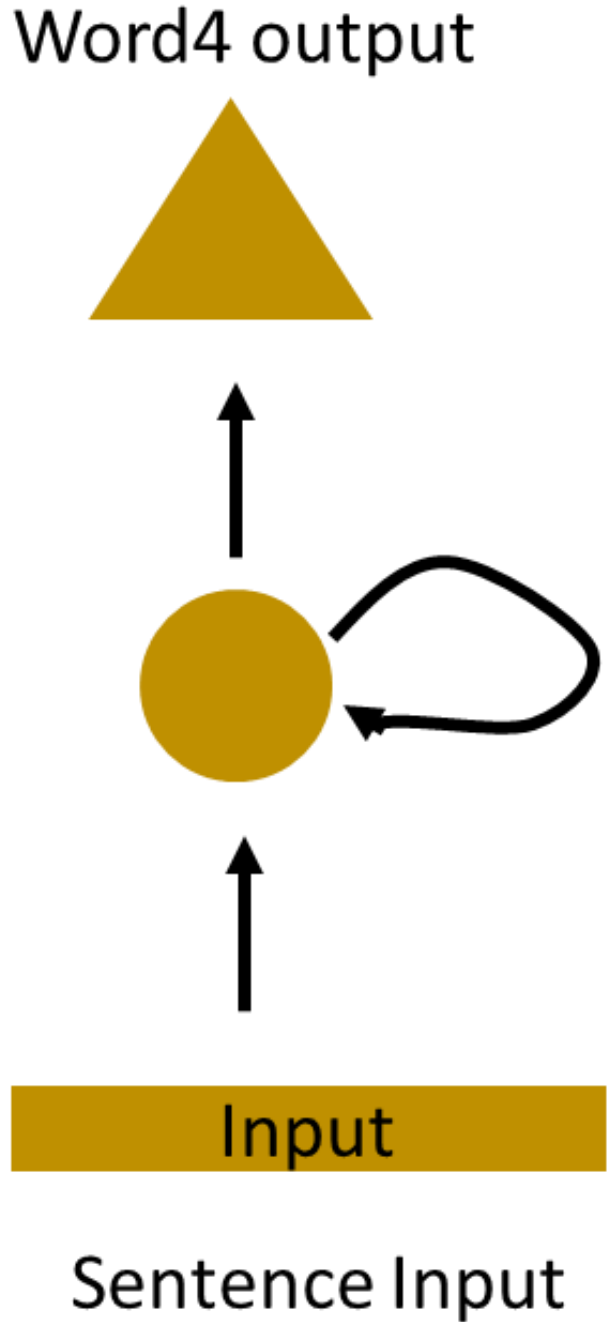
So, what's RNN?

- Multiple hidden layers with different inputs, indeed the previous outputs !!

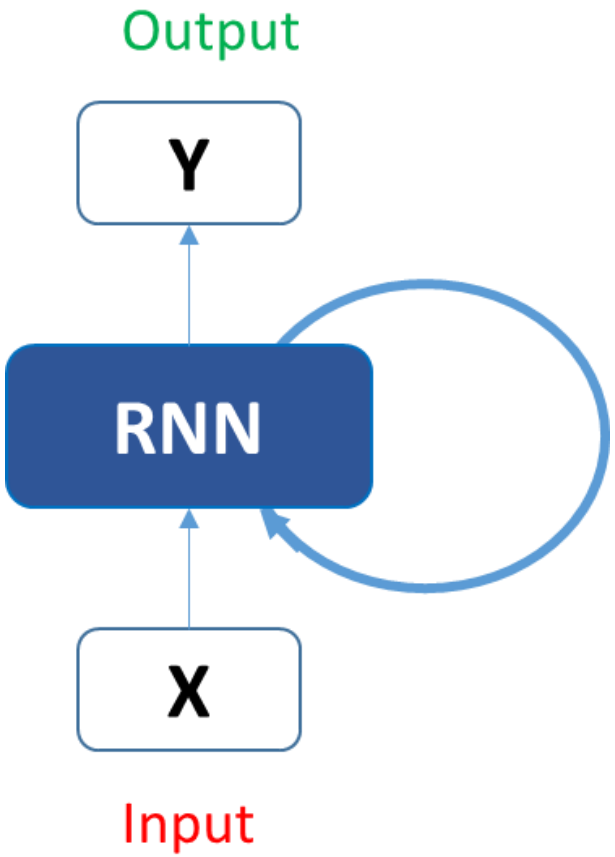


So, what's RNN?

- Hidden layers rolled in,



Recurrent Neural Network



$$h_t = f(h_{t-1}, x_t)$$

$$h_t = \tanh\left(W_{hh}h_{t-1} + W_{xh}x_t\right)$$

$$y_t = W_{hy}h_t$$

Recurrent Neural Network

```
state_t = 0                          <---- The state at t
for input_t in input_sequence:       <---- Iterates over sequence elements
    output_t = f(input_t, state_t)
    state_t = output_t               <---- The previous output becomes the state for the next iteration.
```

```
state_t = 0
for input_t in input_sequence:
    output_t = activation(dot(W, input_t) + dot(U, state_t) + b)
    state_t = output_t
```

Listing 6.21 Numpy implementation of a simple RNN

```
import numpy as np

timesteps = 100
input_features = 32
output_features = 64

inputs = np.random.random((timesteps, input_features))

state_t = np.zeros((output_features,))

W = np.random.random((output_features, input_features))
U = np.random.random((output_features, output_features))
b = np.random.random((output_features,))

successive_outputs = []
for input_t in inputs:
    output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b)

    successive_outputs.append(output_t)

    state_t = output_t

final_output_sequence = np.concatenate(successive_outputs, axis=0)
```

Number of timesteps in the input sequence

Dimensionality of the input feature space

Dimensionality of the output feature space

Input data: random noise for the sake of the example

Initial state: an all-zero vector

Creates random weight matrices

input_t is a vector of shape (input_features,).

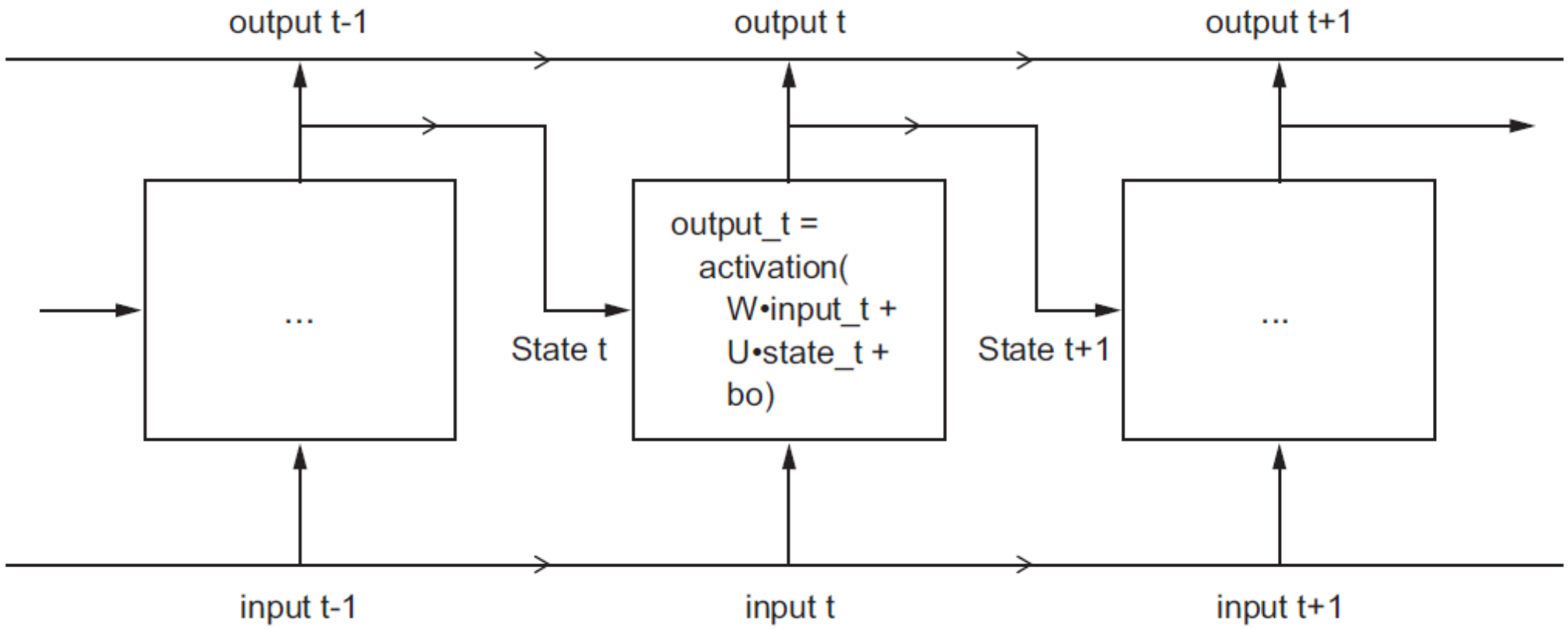
Stores this output in a list

Combines the input with the current state (the previous output) to obtain the current output

The final output is a 2D tensor of shape (timesteps, output_features).

Updates the state of the network for the next timestep

Recurrent Neural Network



RNN in Keras

```
>>> from keras.models import Sequential
>>> from keras.layers import Embedding, SimpleRNN
>>> model = Sequential()
>>> model.add(Embedding(10000, 32))
>>> model.add(SimpleRNN(32))
>>> model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding_22 (Embedding)     (None, None, 32)          320000
_________________________________________________________________
simplernn_10 (SimpleRNN)     (None, 32)                2080
=================================================================
Total params: 322,080
Trainable params: 322,080
Non-trainable params: 0
```

RNN in Keras: Return Full Sequence

```
>>> model = Sequential()
>>> model.add(Embedding(10000, 32))
>>> model.add(SimpleRNN(32, return_sequences=True))
>>> model.summary()
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding_23 (Embedding)     (None, None, 32)          320000
_________________________________________________________________
simplernn_11 (SimpleRNN)     (None, None, 32)          2080
=================================================================
Total params: 322,080
Trainable params: 322,080
Non-trainable params: 0
```

RNN in Keras: Stack RNNs

```
>>> model = Sequential()
>>> model.add(Embedding(10000, 32))
>>> model.add(SimpleRNN(32, return_sequences=True))
>>> model.add(SimpleRNN(32, return_sequences=True))
>>> model.add(SimpleRNN(32, return_sequences=True))
>>> model.add(SimpleRNN(32))
>>> model.summary()
```

Last layer only returns the last output

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding_24 (Embedding)     (None, None, 32)          320000
_________________________________________________________________
simplernn_12 (SimpleRNN)     (None, None, 32)          2080
_________________________________________________________________
simplernn_13 (SimpleRNN)     (None, None, 32)          2080
_________________________________________________________________
simplernn_14 (SimpleRNN)     (None, None, 32)          2080
_________________________________________________________________
simplernn_15 (SimpleRNN)     (None, 32)                2080
=================================================================
Total params: 328,320
Trainable params: 328,320
Non-trainable params: 0
```

Code: IMDB on RNN

- Accuracy in first chapter on IMDB using Dense Fully Connected was 88%
- Accuracy using RNN is 85.5%
- Why?????

Code: IMDB on RNN

- Accuracy in first chapter on IMDB using Dense Fully Connected was 88%
- Accuracy using RNN is 85.5%
- Why?????
  - RNN has access to only 500 words in review whereas Dense Fully Connected has access to complete review
  - Re-Run with review size 1000 and max_features to 25,000
  - What is performance now?

Code: IMDB on RNN (IMDB_RNN)

- Accuracy in first chapter on IMDB using Dense Fully Connected was 88%
- Accuracy using RNN is 85.5%
- Why?????
  - RNN has access to only 500 words in review whereas Dense Fully Connected has access to complete review
  - Re-Run with review size 1000 and max_features to 25,000
  - What is performance now?
- SimpleRNN is not good processing long sequences
- LSTM is next big thing…

LSTM and GRU

- Both are RNNs

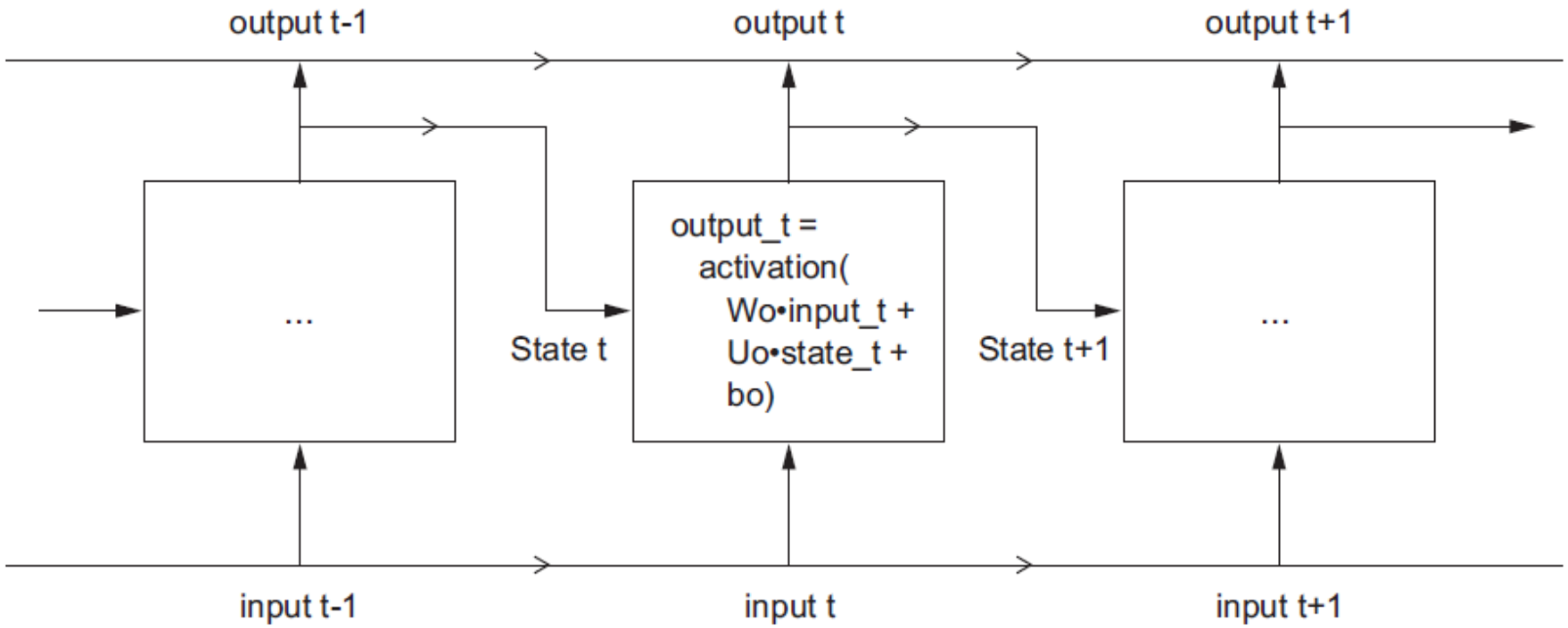


Figure 6.13 The starting point of an `LSTM` layer: a `SimpleRNN`

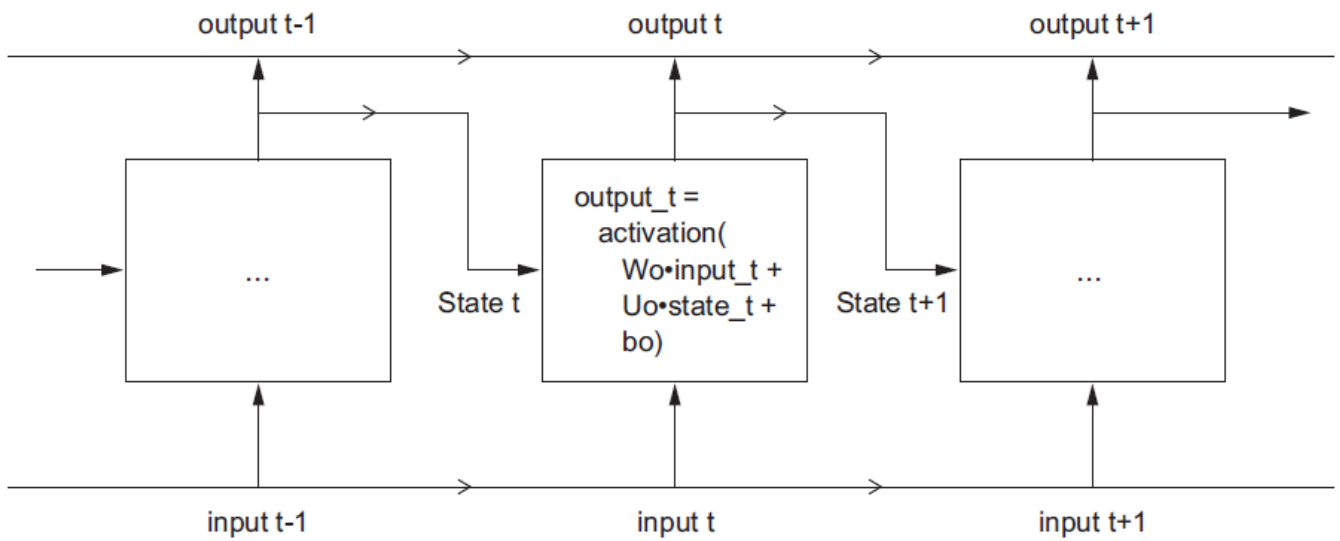


Figure 6.13 The starting point of an `LSTM` layer: a `SimpleRNN`

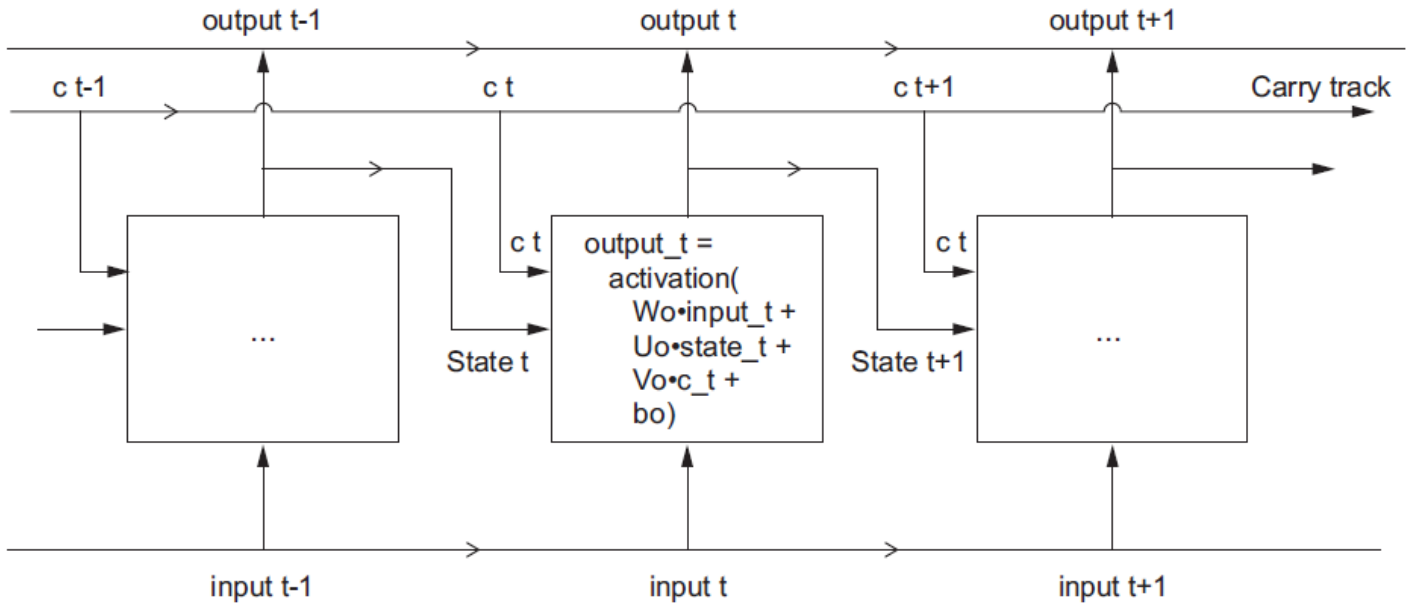


Figure 6.14 Going from a `SimpleRNN` to an `LSTM`: adding a carry track

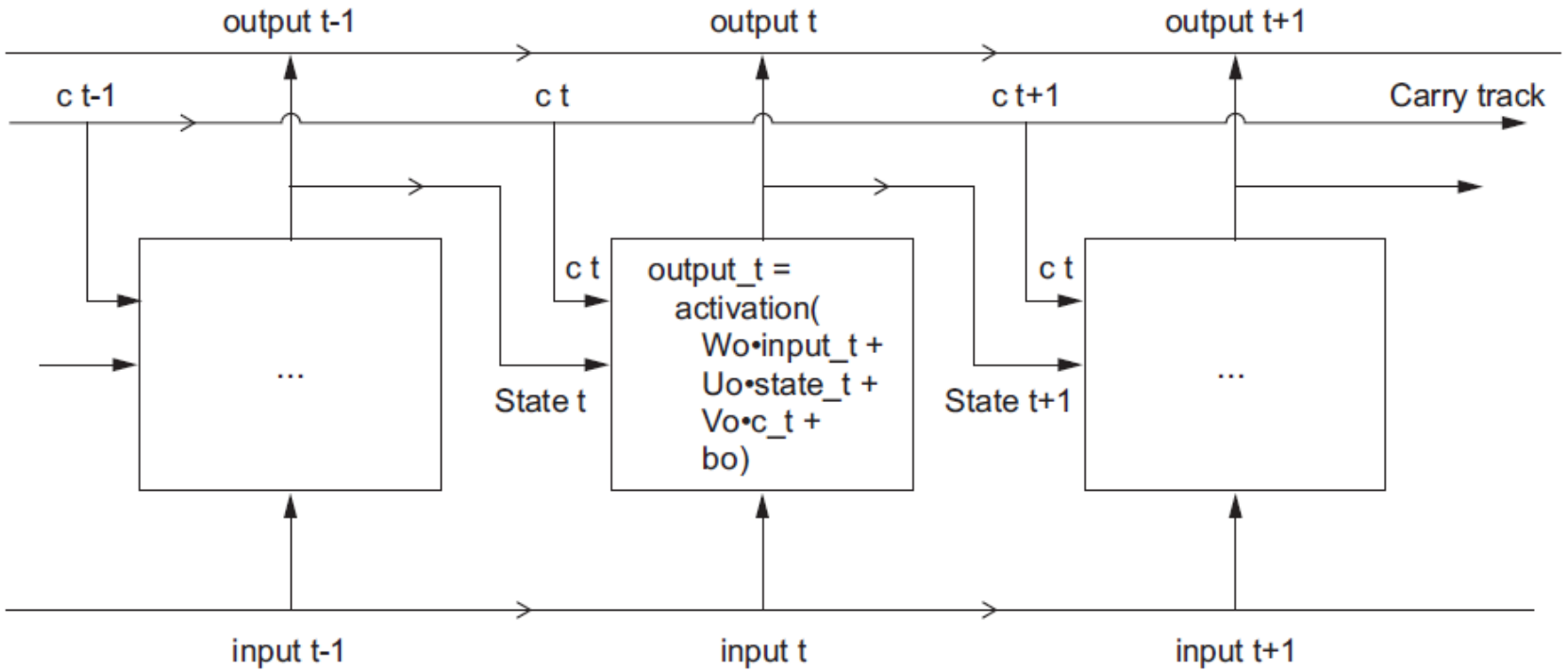


Figure 6.14 Going from a `SimpleRNN` to an `LSTM`: adding a carry track

```
output_t = activation(dot(state_t, Uo) + dot(input_t, Wo) + dot(C_t, Vo) + bo)

i_t = activation(dot(state_t, Ui) + dot(input_t, Wi) + bi)
f_t = activation(dot(state_t, Uf) + dot(input_t, Wf) + bf)
k_t = activation(dot(state_t, Uk) + dot(input_t, Wk) + bk)
```

```
c_t+1 = i_t * k_t + c_t * f_t
```

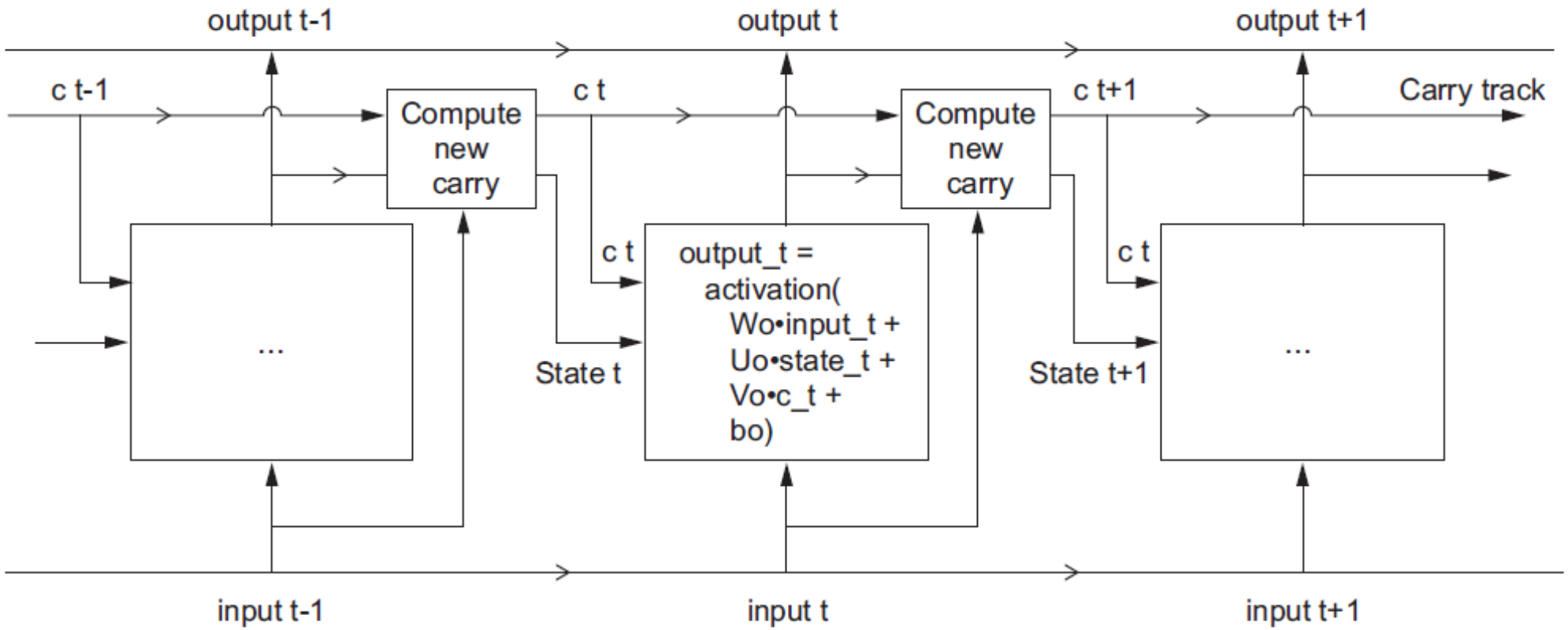


Figure 6.15 Anatomy of an LSTM

Code: LSTM_IMDB

Bidirectional RNN

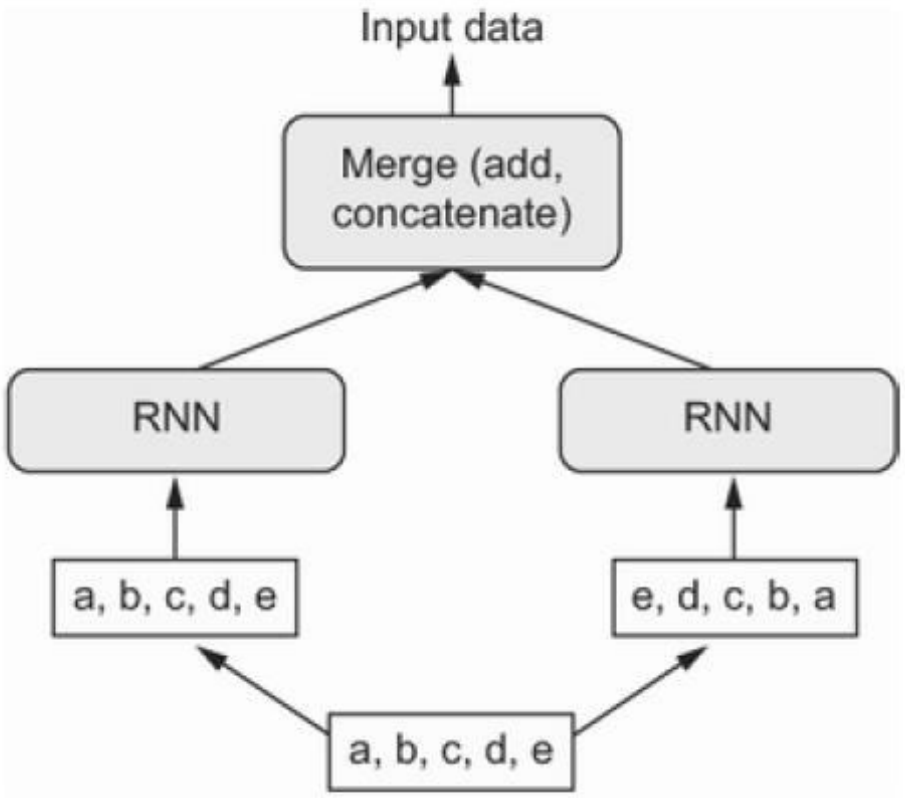


Figure 10.14 How a bidirectional RNN layer works

Two approaches of text processing

- Set
- Sequence

Experiments

- TextVectorization layer (set approach)
  - multi_hot
  - multi_hot with ngram=2
  - Bigram with tf_idf
- LSTM: sequence approach
  - one_hot embedding (takes ages)
  - Embedding layer to learn word weights
  - Pre-trained word embedding (GloVe)

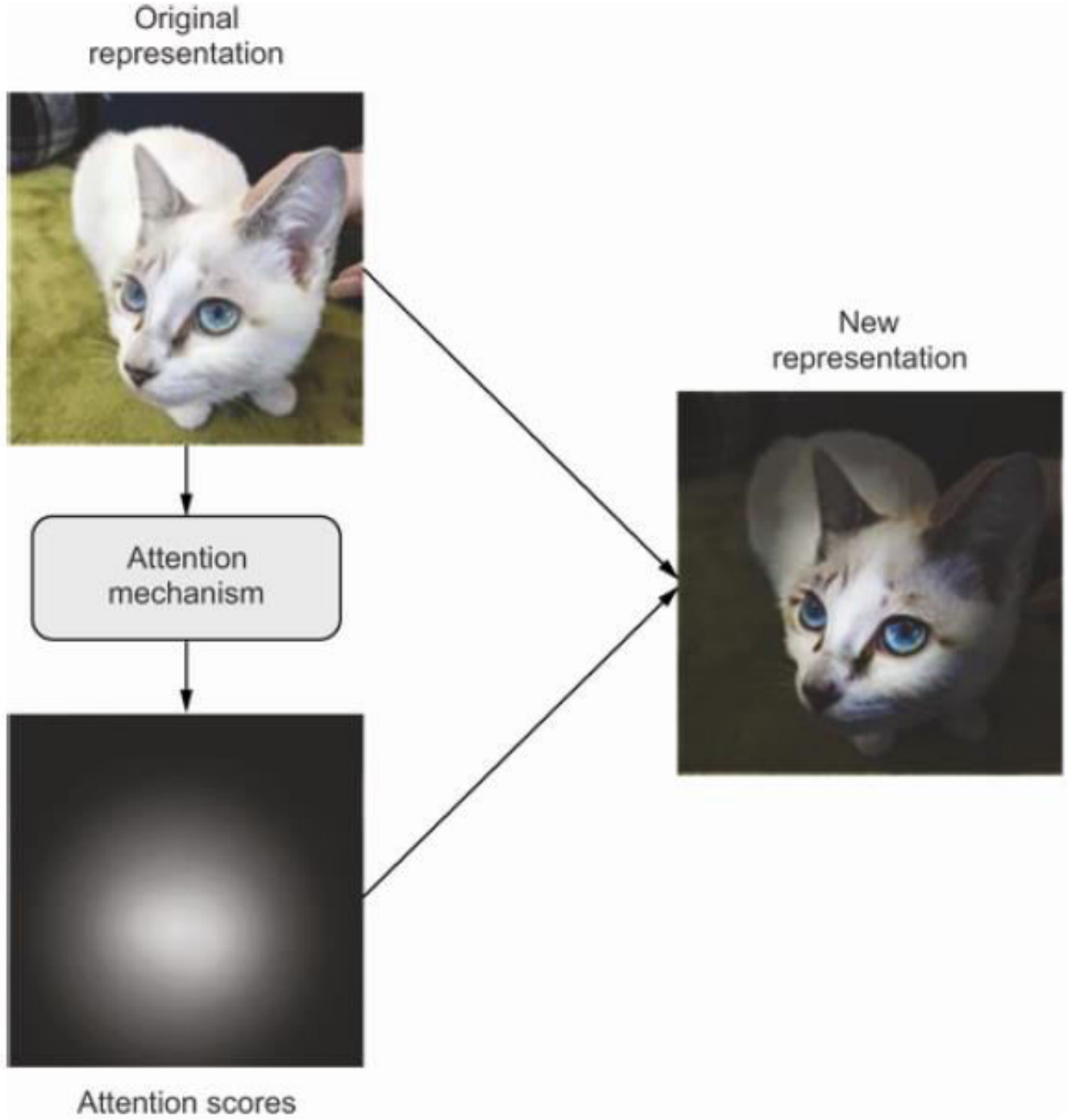
The concept of transformer

- Sequence matters
- Originally a seq-2-seq model
- Attention concept is at the core of transformers
  - Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).
- BERT, GPT models are results of transformer concept

Understanding self-attention

- Two examples of attention,
  - **Max pooling** in convnets looks at a pool of features in a spatial region and selects just one feature to keep. That's an "all or nothing" form of attention: keep the most important feature and discard the rest.
  - **TF-IDF** normalization assigns importance scores to tokens based on how much information different tokens are likely to carry. Important tokens get boosted while irrelevant tokens get faded out. That's a continuous form of attention.

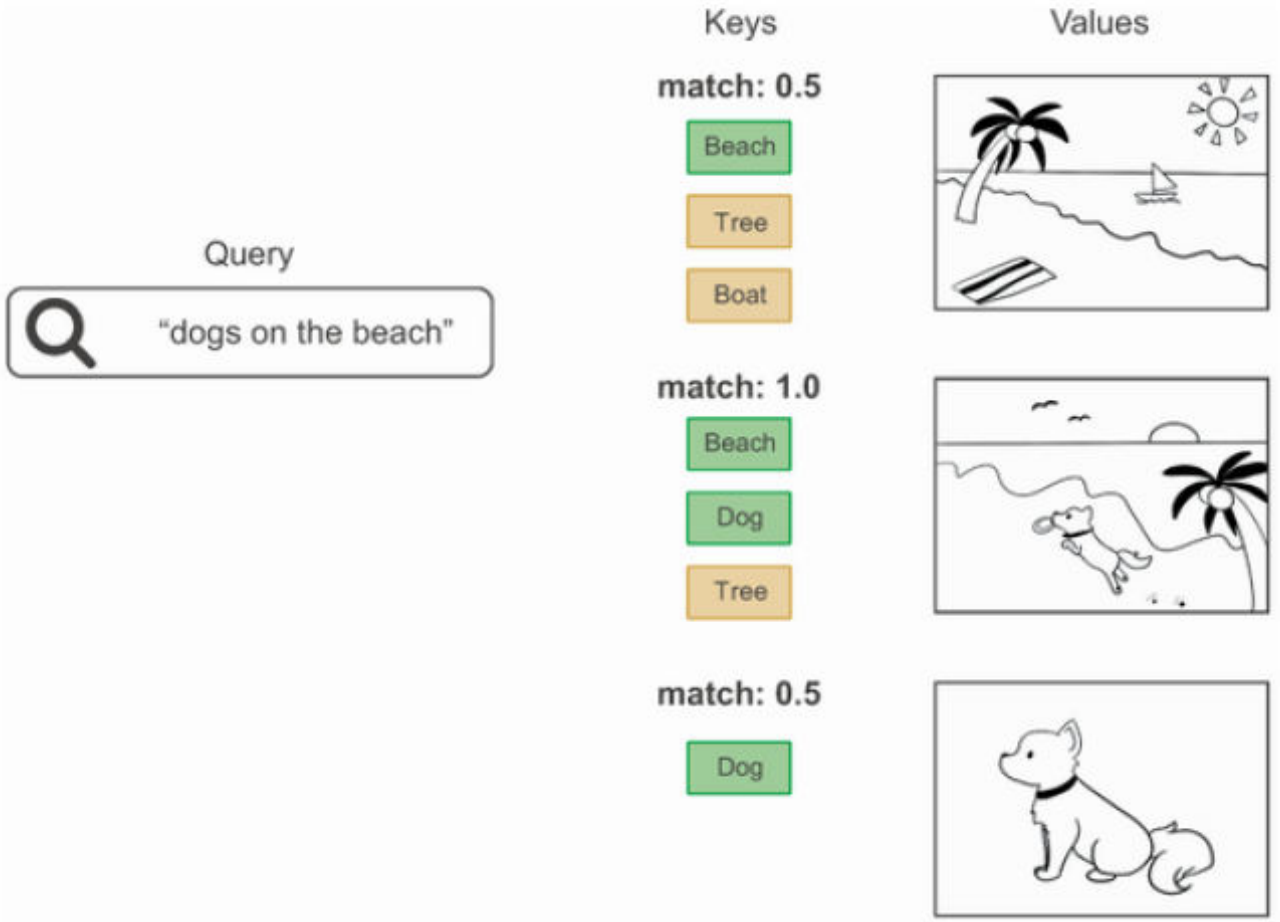
How we pay attention?



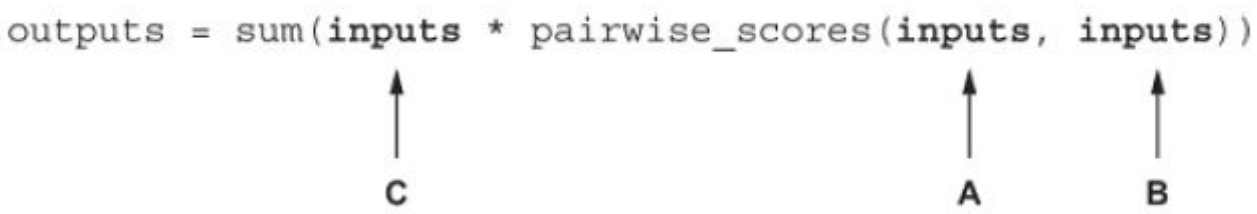
Context Matters!!!

- I will see you
- I see
- I see an apple on a table

Query-Value-Key concept

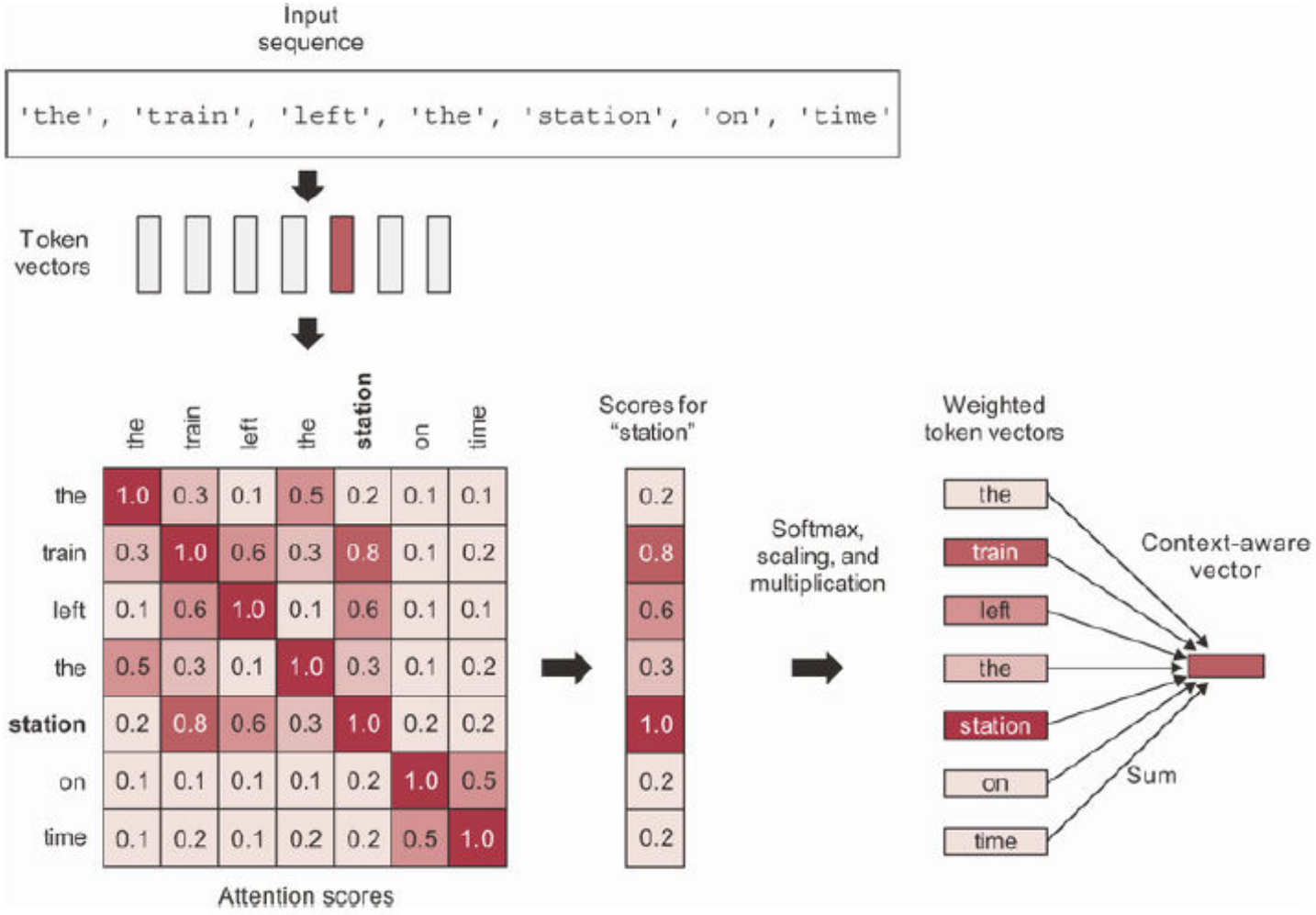


Query-Value-Key concept

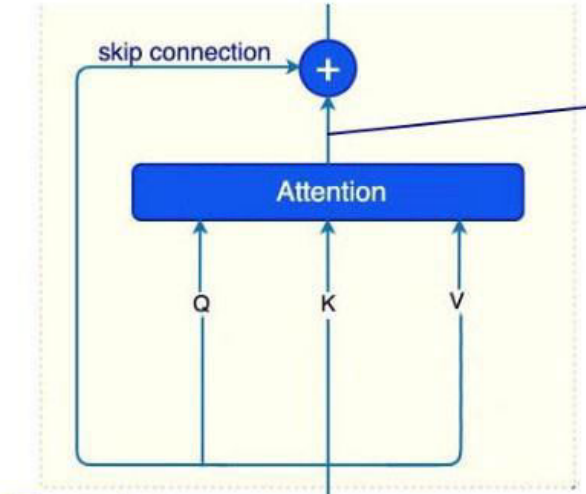



```
outputs = sum(values * pairwise_scores(query, keys))
```

Understanding attention mechanism



Understanding attention mechanism



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

13×512 512×13 13×512 512 13×512

$Q = K = V =$ 13 512-D word embedding vectors for Encoder$_1$

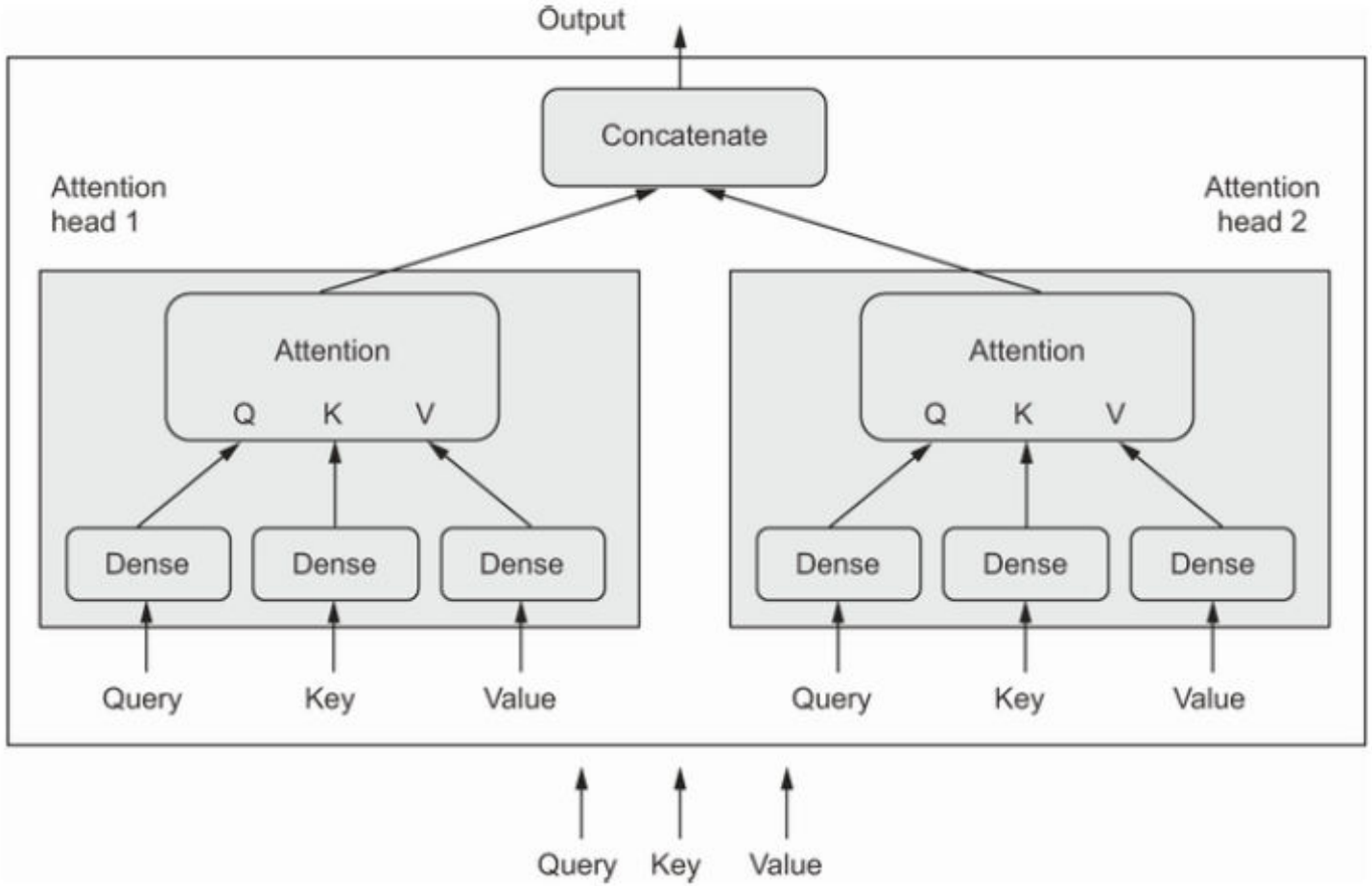
Understanding attention mechanism

```
def self_attention(input_sequence):
    output = np.zeros(shape=input_sequence.shape)
    for i, pivot_vector in enumerate(input_sequence):                    1
        scores = np.zeros(shape=(len(input_sequence),))
        for j, vector in enumerate(input_sequence):
            scores[j] = np.dot(pivot_vector, vector.T)                   2
        scores /= np.sqrt(input_sequence.shape[1])                       3
        scores = softmax(scores)                                         3
        new_pivot_representation = np.zeros(shape=pivot_vector.shape)
        for j, vector in enumerate(input_sequence):
            new_pivot_representation += vector * scores[j]               4
        output[i] = new_pivot_representation                             5
    return output
```

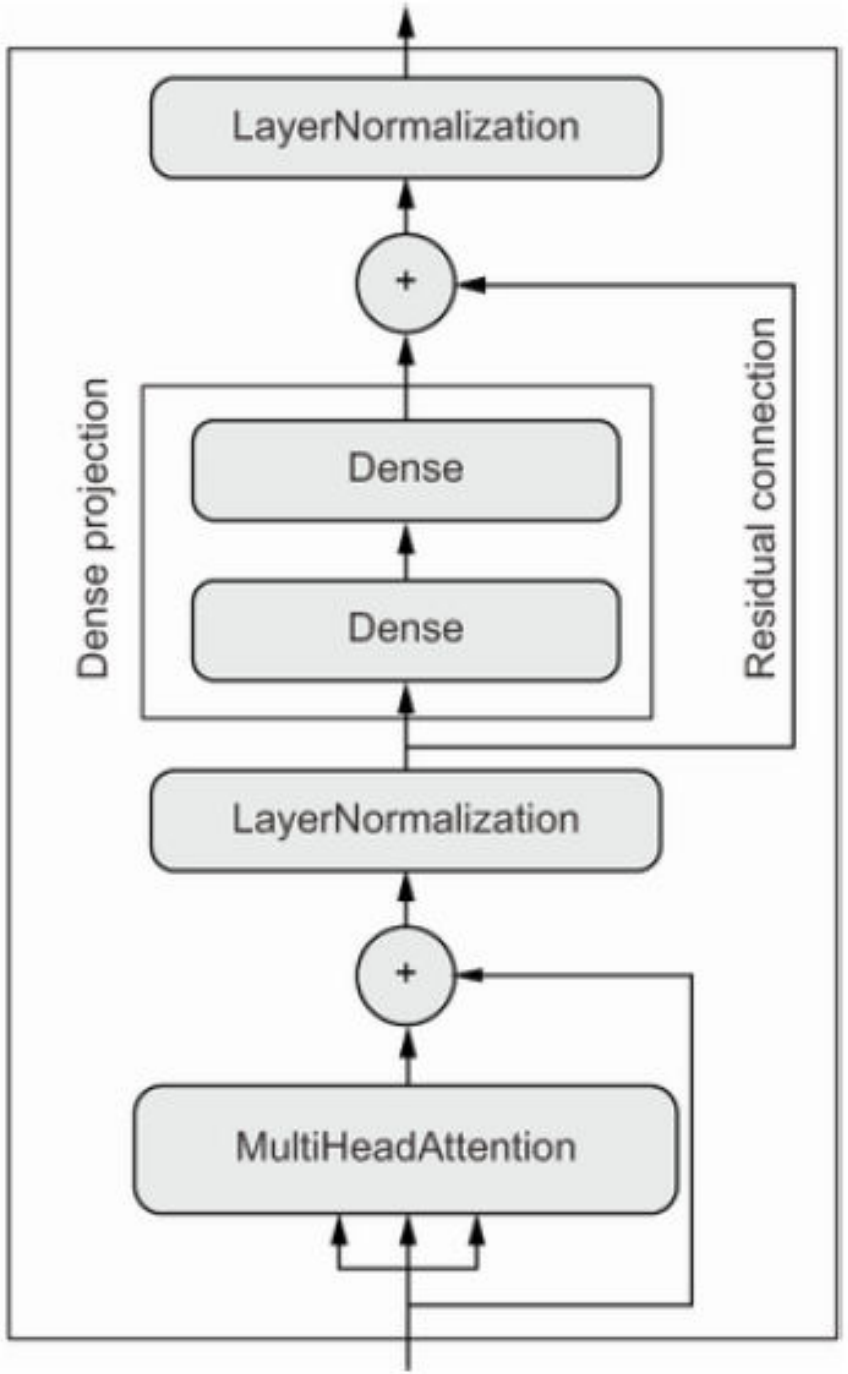
MultiHeadAttention

```
num_heads = 4
embed_dim = 256
mha_layer = MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
outputs = mha_layer(inputs, inputs, inputs)
```

MultiHeadAttention



Transformer Encoder



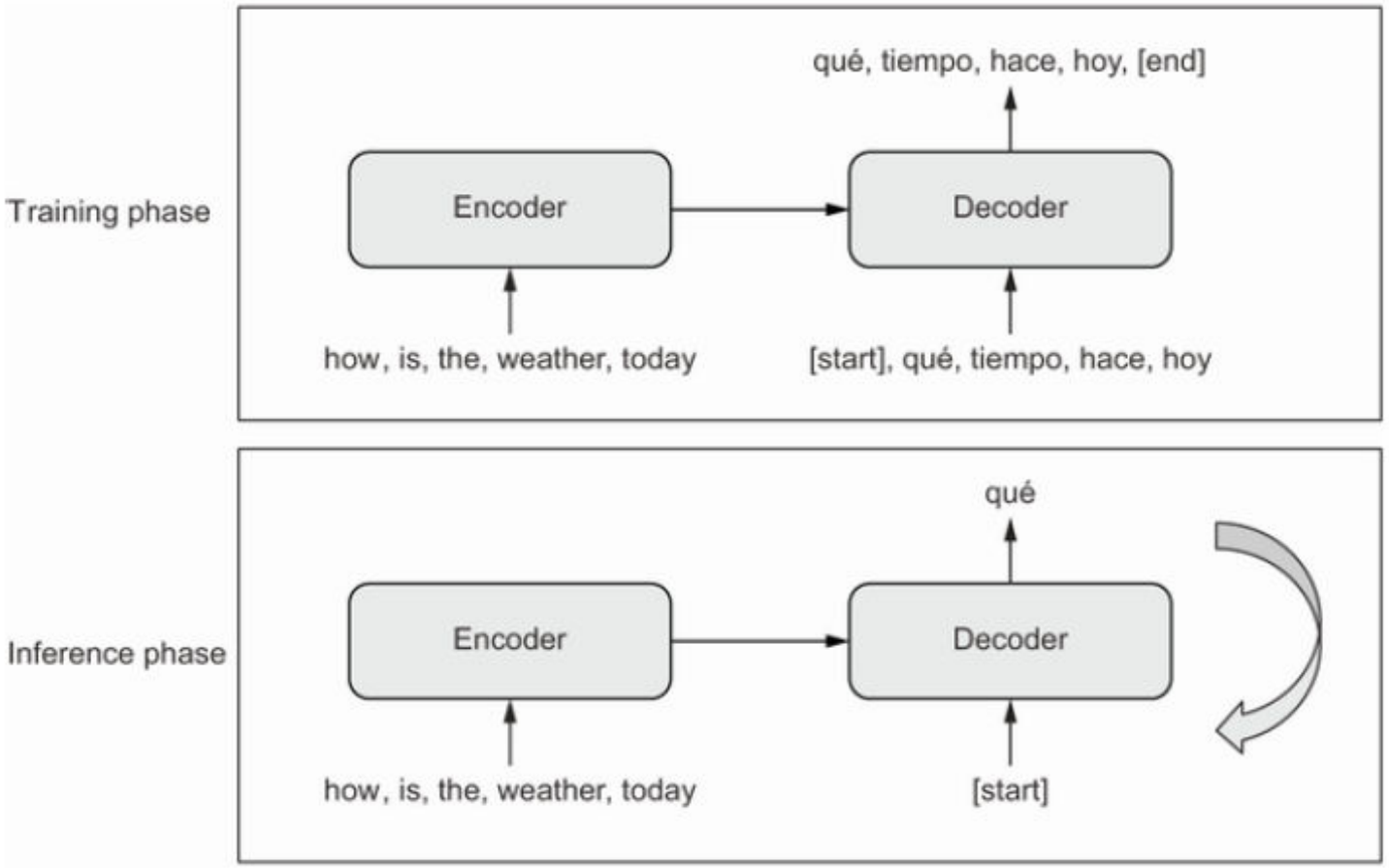
Implementation (chapter11_part03_transformer)

Beyond Text Classification: seq2seq learning

- Machine Translation
- Text Summarization
- Question Answering
- Chatbots
- Text generation

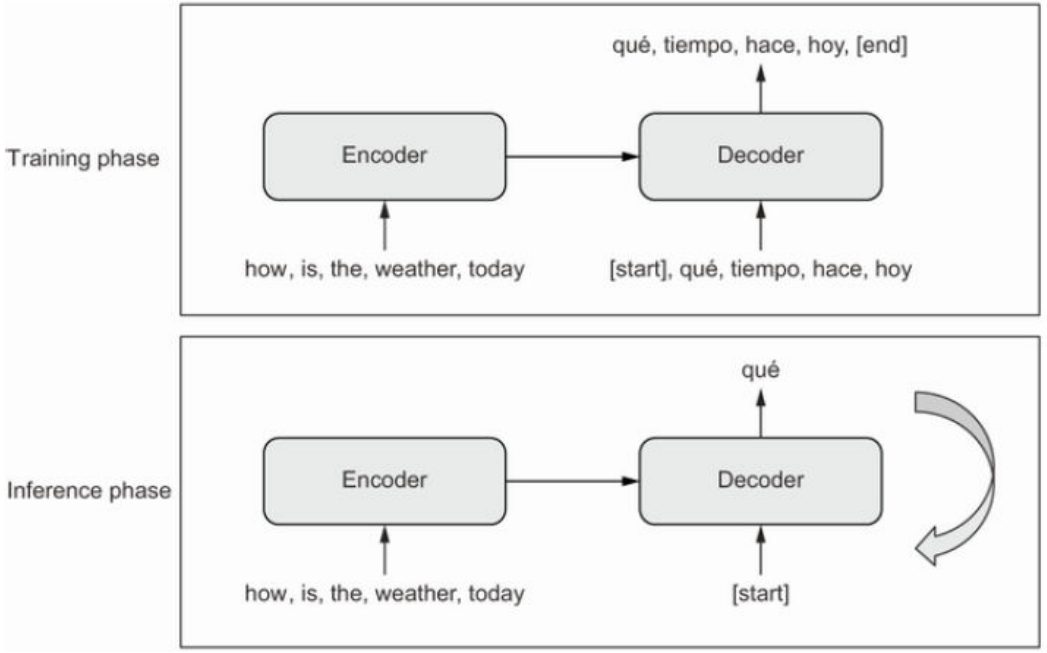
Beyond Text Classification: seq2seq learning

- General Model



Beyond Text Classification: seq2seq learning

1. We obtain the encoded source sequence from the encoder.
2. The decoder starts by looking at the encoded source sequence as well as an initial "seed" token (such as the string "`[start]`"), and uses them to predict the first real token in the sequence.
3. The predicted sequence so far is fed back into the decoder, which generates the next token, and so on, until it generates a stop token (such as the string "`[end]`").



Implementation chapter11_part04_transformer